GloVe: Global Vectors for Word Representation

Combining count-based and prediction-based embeddings

Pennington, Socher & Manning (2014)

Roadmap

1. Recap: count-based vs. prediction-based word representations
2. The idea behind GloVe: the best of both worlds
3. Intuition: ratios of co-occurrence probabilities (*ice* vs. *steam*)
4. Deriving the objective: dot products, biases, squared loss
5. The weighting function $f(X_{ij})$
6. Applications: downstream tasks, analogies, bias, meaning over time

Recap: two families of methods

Word representations so far

Count-based ("old school")

- Term–context matrix
- TF-IDF weighted matrix
- PMI / PPMI (pointwise mutual information, positive PMI)

Prediction-based

- word2vec
- Slide a context window over the text and train a classifier (logistic regression) for each window

This deck

Another embedding method — **GloVe** (Global Vectors) — and some important properties of word embeddings.

Count-based methods: pros and cons

Advantages

- **Fast to train**: scan the corpus once, fill in the co-occurrence matrix, apply a simple computation (e.g. TF-IDF).
- **Uses co-occurrence statistics properly**: how often two words appear together is a statistically important count, and it is captured exactly.

Disadvantages

- Large, **sparse** matrix — most entries are zero; not the small dense vector we want for an embedding.
- Mainly captures **word similarity**.
- Gives **disproportionate importance** to very frequent co-occurrences (e.g. stop-word pairs) — no way to down-weight them.

Prediction-based methods (word2vec): pros and cons

Advantages

- **Improved performance on downstream tasks** (sentiment analysis, named entity recognition, ...).
- Dense vectors $\Rightarrow$ less likely to overfit.
- Captures **complex patterns** through linear operations: $king - man + woman \approx queen$.

Disadvantages

- **Scales with corpus size**: every new token means more window positions and more classifier updates.
- Does **not use global statistics** — never looks directly at how many times two words co-occur.

Scaling: fixed vocabulary, growing corpus

Add 100 new tokens, *no* new vocabulary words

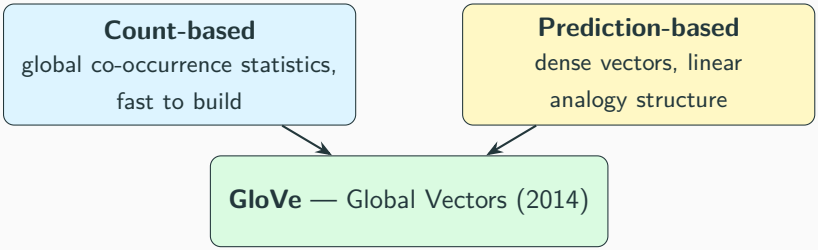
Count-based
matrix stays $|V| \times |V|$

Just scan the new text and update counts. Done.

Prediction-based
work grows with #tokens

Slide the window (e.g. size 7) across every new position and re-run the logistic regression for each window.

The best of both worlds

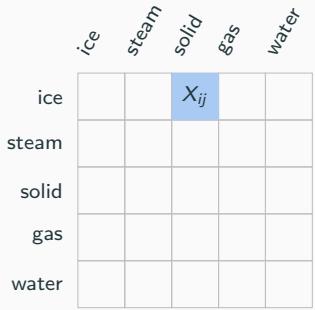


Proposed the year after word2vec by Christopher Manning's group at Stanford: Jeffrey Pennington (postdoc) and Richard Socher (PhD student).

Intuition: ratios of co-occurrence probabilities

Step 1: build a co-occurrence matrix

- Same as the count-based approach: a **words** $\times$ **words** matrix X.
- Fix a window size; each time words i and j appear in the same window, increment X_{ij}.
- But we will not use the raw counts directly.



Step 2: probe two target words with context words

Target words from thermodynamics: **ice** and **steam**. Every vocabulary word can act as a context ("pivot") word; take four:

	k = solid	k = gas	k = water	k = fashion
$P(k \mid ice)$	**high**	**low**	**high**	**low**
$P(k \mid steam)$	**low**	**high**	**high**	**low**

$P(j \mid i) = X_{ij}/X_i$ is read straight off the co-occurrence matrix (i = target, j = context).

Step 3: take the ratio

	solid	gas	water	fashion
$\frac{P(k \mid ice)}{P(k \mid steam)}$	$\frac{\text{high}}{\text{low}}$	$\frac{\text{low}}{\text{high}}$	$\frac{\text{high}}{\text{high}}$	$\frac{\text{low}}{\text{low}}$
	large	**small**	≈ 1	≈ 1

- Ratio far from 1 $\Rightarrow$ the context word is **relevant** for telling ice from steam — and its direction says which one (*solid* $\rightarrow$ ice, *gas* $\rightarrow$ steam).
- Ratio ≈ 1 $\Rightarrow$ **not discriminative**: either related to both (*water*) or to neither (*fashion*).

Why the ratio matters

The ratio tells us a lot

- Separates **important** context words from **less important** ones.
- Says **which target** each important context word belongs to.
- Raw probabilities alone can't do this: *water* has a high probability with ice, yet it is useless for distinguishing ice from steam.

GloVe uses this philosophy to derive its training objective.

Deriving the GloVe objective

Model similarity with log co-occurrence probability

For words i, j we want to learn embeddings $\mathbf{w}_i, \mathbf{w}_j$ whose **dot product** reflects the measured probability:

$$\mathbf{w}_i^\top \mathbf{w}_j \;=\; \log P(j \mid i) \;=\; \log \frac{X_{ij}}{X_i} \;=\; \log X_{ij} - \log X_i$$

X_{ij}: number of times i and j co-occur; X_i: count for word i (sum of row i).

Problem: asymmetry

$P(j \mid i) \neq P(i \mid j)$, but $\mathbf{w}_i^\top \mathbf{w}_j = \mathbf{w}_j^\top \mathbf{w}_i$. Swapping target and context gives a second equation: $\mathbf{w}_j^\top \mathbf{w}_i = \log X_{ij} - \log X_j$

Symmetrise by adding the two equations

$$\mathbf{w}_i^\top \mathbf{w}_j = \log X_{ij} - \log X_i$$
$$\mathbf{w}_j^\top \mathbf{w}_i = \log X_{ij} - \log X_j$$

$$2\,\mathbf{w}_i^\top \mathbf{w}_j = 2\log X_{ij} - \log X_i - \log X_j$$

$$\boxed{\mathbf{w}_i^\top \mathbf{w}_j \;=\; \log X_{ij} \;-\; \tfrac{1}{2}\log X_i \;-\; \tfrac{1}{2}\log X_j}$$

The last two terms **do not depend on the co-occurrence** of i and j — each depends only on a single word.

Absorb word-only terms into biases

Treat each word-only term as a learned property of that word — a **bias**:

$$-\tfrac{1}{2}\log X_i \longrightarrow b_i \qquad\qquad -\tfrac{1}{2}\log X_j \longrightarrow b_j$$

$$\mathbf{w}_i^\top \mathbf{w}_j + b_i + b_j \approx \log X_{ij}$$

- The biases are learned along with the vectors.
- They can also be dropped: $\mathbf{w}_i^\top \mathbf{w}_j \approx \log X_{ij}$.

Turn it into a squared loss

Goal: find vectors whose dot product (plus biases) mimics the count-based quantity $\log X_{ij}$. Minimise the squared difference over all pairs:

$$J = \sum_{i,j} \left(\mathbf{w}_i^\top \mathbf{w}_j + b_i + b_j - \log X_{ij} \right)^2$$

Still a problem

Very large X_{ij} (e.g. stop-word pairs) still get a **disproportionate** say in the loss — the same weakness as count-based methods.

The weighting function

Add a weight $f(X_{ij})$ to each term

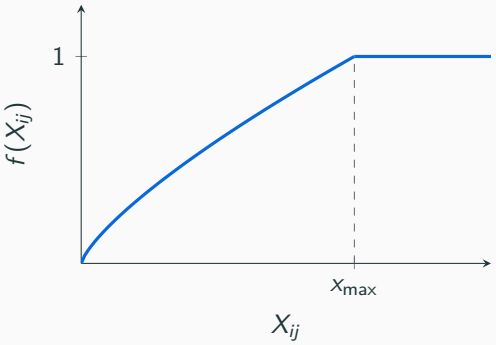
$$J = \sum_{i,j} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Choose any f satisfying three properties:

1. **Continuous**, and stays finite as $x \to 0$.
2. **Non-decreasing**: never decreases as co-occurrence grows.
3. **Relatively small for large** x: very frequent pairs must not get proportionally large weight.

f penalises pairs whose co-occurrence is too high — e.g. stop-word pairs — relative to their raw count.

The GloVe weighting function



- A **cutoff** $x_{\max}$.
- Below $x_{\max}$: rises almost linearly with the count.
- Above $x_{\max}$: weight is capped at **1** — very frequent pairs get no extra weight.
- Any other f meeting the three properties would also work.

The final GloVe objective

$$J = \sum_{i,j} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- Minimise with gradient descent: take derivatives with respect to $\mathbf{w}_i$ and $\tilde{\mathbf{w}}_j$.
- As in word2vec, **two embeddings** are learned per word: one as target ($\mathbf{w}$), one as context ($\tilde{\mathbf{w}}$).

From count-based

$\log X_{ij}$ — global co-occurrence statistics

From prediction-based

$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j$ — dense vectors learned by gradient descent

GloVe in practice

- A **static** embedding: one vector per word type.
- Showed strong results across multiple tasks.
- **Fast to train** and **scalable** to huge corpora.
- Also performs well with a **small corpus** — unlike word2vec.

Further reading: the original paper (Pennington, Socher & Manning, EMNLP 2014).

What embeddings let you do

Applications of GloVe

Features for downstream models

- Sentiment, NER, POS tagging, text classification
- Initialise LSTM/CNN embedding layers (helps with little labelled data)

Similarity & retrieval

- Nearest neighbours: query expansion, synonyms
- Averaged vectors as cheap sentence vectors

Probing language

- Analogies and relation completion
- Measuring and reducing corpus bias
- Tracking meaning change over time

Limitation

One static vector per word: *bank* (river) = *bank* (money). Contextual models (BERT) fix this; GloVe stays a fast baseline.

Linear analogies

Beyond feeding downstream tasks, embeddings support interesting studies.



- *Beijing* is to *China* as *Delhi* is to $__ \rightarrow$ **India**
- $king - man + woman \approx queen$

Embeddings inherit bias

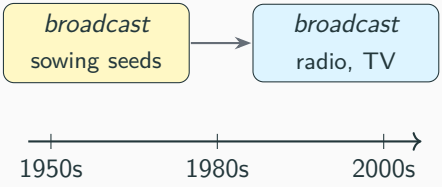
The same analogy mechanism exposes biases learned from the corpus:

Query		Returned
father : *computer programmer* :: *mother* : ?	$\rightarrow$	homemaker
man : *doctor* :: *woman* : ?	$\rightarrow$	nurse

Methods like word2vec and GloVe are **prone to bias** present in their training text.

Tracking meaning over time

1. Take e.g. New York Times articles, 1950–2010.
2. Split by decade; train a separate GloVe (or word2vec) embedding on each.
3. Pick a word and compare its **nearest neighbours** decade by decade.



The meaning of *broadcast* drifted from agriculture to radio and frequencies.

Summary

- Count-based: fast, statistically faithful, but sparse and dominated by frequent pairs. Prediction-based: dense and analogy-friendly, but cost grows with the corpus and ignores global counts.
- GloVe builds a co-occurrence matrix and learns vectors so that $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \approx \log X_{ij}$.
- Motivation: **ratios** of co-occurrence probabilities separate relevant from irrelevant context words.
- A weighting $f(X_{ij})$, capped at $x_{\max}$, stops frequent pairs from dominating.
- Embeddings enable analogies and diachronic studies — and reflect corpus bias.

Next: word embeddings as inputs to neural language models and Transformers.